LIFELINE: Algorithm-selection report (proposal), Iteration 1. Group 18: Saneed Ullah (23i-2568), Hasnat Ahmed (23i-2652), Hussain Muhammad Ibrahim (23i-2626), AI-B				
Module	**Problem (n, m, V, E, W)**	**Candidates (time complexity)**	**Selection**	**Justification (data property, assumption, failure case)**
A. Relief inventory	**In:** n crates (weight w_i 1..20 kg, value v_i = floor(base × day multiplier)), capacity W. **Out:** a set S of whole crates with $\Sigma w_i \le W$. **Objective:** maximise Σv_i. This is the 0/1 knapsack problem (NP-hard in general, pseudo-polynomial in W).	1. Exhaustive search over all subsets: $O(2^n \cdot n)$. 2. DP over capacity: $O(n \cdot W)$ time, $O(n \cdot W)$ bits to rebuild the load. 3. Greedy by value/weight, exact fractions: $O(n \log n)$, no guarantee. 4. Branch & bound with a fractional bound: $O(2^n)$ worst case.	**Primary:** DP over capacity with a keep-table to rebuild the chosen crates. **Baseline:** greedy by value/weight (ties by crate_id). Exhaustive search is the exactness reference on tier S against cargo_opt.	**Data property:** weights are small integers and W ≤ 70% of the total weight. In our cases W ≤ 3,388 and $n \cdot W \le 4.2 \times 10^6$ cells (case 18-45), so the DP is cheap. Day multipliers change values, not weights, so the table size stays the same every day. **Assumption:** integer weights. **Fails when:** W is very large (master data with n = 3,000 and W of about 40,000 gives 1.2×10^8 cells), so the memory needed to rebuild the load becomes the bottleneck; branch & bound is the fallback. Greedy fails on trap 47 (55% of optimum).
B. Route planning	**In:** map G = (V, E), source HQ = 0, today's closed roads (waves). Directed step cost c(u→v) = minutes(u,v) + penalty(v). **Out:** a cheapest route to the target and its cost, or "no route"; the set of reachable locations; the fewest-roads route (BFS, neighbours in ascending id). **Objective:** minimise the route cost.	1. BFS: $O(V + E)$ (reachability, fewest roads). 2. Dijkstra with a binary heap: $O((V + E) \log V)$. 3. Dijkstra with an array: $O(V^2)$. 4. Bellman-Ford: $O(V \cdot E)$.	**Primary:** Dijkstra with a binary heap (lazy deletion) for cost; BFS for reachable locations and the fewest-roads route. **Baseline:** Bellman-Ford (exact, slower), plus the BFS route used as if it were the cheapest (the naive choice).	**Data property:** every step costs at least 1 (minutes ≥ 1, penalty ≥ 0, and both the storm ×1.5 rounded up and the doubled penalties keep it positive), so Dijkstra is correct. Maps are sparse ($E \le 3V$; ours have V ≤ 2,140 and E ≤ 2,867), so the heap beats the $O(V^2)$ array. **Assumption:** no negative costs, and the penalty belongs to the destination (costs are directed). **Fails when:** a negative cost appears (then Bellman-Ford is needed). The BFS route fails on trap 48 (≥ 1.5× the cheapest cost).
C. Rescue scheduling	**In:** n windows $[s_i, f_i)$ in minutes 0..1440, value v_i. Windows i and j are compatible if $f_i \le s_j$ or $f_j \le s_i$ (touching is allowed). **Out:** a set of pairwise compatible windows. **Objective:** (a) the most windows; (b) the most total value.	1. Exhaustive search: $O(2^n \cdot n)$. 2. Earliest-finish greedy: $O(n \log n)$. 3. Greedy by highest value: $O(n \log n)$, no guarantee. 4. Weighted interval DP + binary search: $O(n \log n)$; with a linear scan: $O(n^2)$.	**Primary:** (a) earliest-finish greedy (provably optimal for the count); (b) weighted interval DP with binary search on finish times. **Baseline:** greedy by highest value, and the $O(n^2)$ DP.	**Data property:** windows lie in one day with integer ends; n ≤ 3,000 (ours ≤ 2,875). Study 3 is dense (≥ 2 overlaps per window), which is where greedy-by-value does badly. **Assumption:** one rescue team (one window at a time), and touching windows are compatible, so the test is $s_j \ge f_i$ (not >). Ties go by finish, then window_id. **Fails when:** several teams can work at once (this becomes k-machine scheduling), or windows need set-up or travel time between them.
D. Emergency queue	**In:** m alarms (tick t_i, priority p_i 1..10, alarm_id). **Out:** the release order. At each tick, every alarm that has arrived joins the queue and exactly one is released. **Objective:** always release the most urgent alarm present (priority descending, then tick ascending, then alarm_id ascending).	1. Binary max-heap: $O(m \log m)$. 2. Sorted array with insertion: $O(m^2)$ shifts. 3. Unsorted list, scanned for the max: $O(m^2)$. 4. Bucket queue (one queue per priority): $O(m + 10 \cdot T)$.	**Primary:** binary heap keyed on (−p, t, id). **Baseline:** sorted array with insertion (also measured: the unsorted scan and the bucket queue).	**Data property:** alarms arrive at 1.2 to 1.8 per tick (ours 1.27 to 1.75) but only 1 is released per tick, so the queue grows linearly: in our data up to 1,826 alarms wait at once (case 18-47, m = 4,513). The list versions turn quadratic while the heap stays $O(\log m)$ per operation. **Assumption:** a priority never changes while an alarm waits. **Fails when:** priorities age, which needs decrease-key (an indexed heap). With only 10 priorities and arrivals already in (tick, id) order, the bucket queue may beat the heap; we will measure it.

LIFELINE: Work division, Iteration 1. Group 18: Saneed Ullah (23i-2568), Hasnat Ahmed (23i-2652), Hussain Muhammad Ibrahim (23i-2626), AI-B. Every member can explain the whole submission.	
Member (name, roll number, section)	**Contribution in Iteration 1 and decision modules proposed for Iteration 2**
Saneed Ullah 23i-2568 AI-B	**Data modules (generator):** locations, roads and closure waves (spanning tree plus extra roads to each study's average degree, corridor bridges for study 5), sites and the planted tour (nearest neighbour + 2-opt). Trap 48 (shortest-looks-cheapest) and variety rule 2. **Validator checks:** V5 radio pattern counts (KMP) and edit_bound (DP); V8 traps 46 and 49. **Claims:** tour_opt (exhaustive search over tours). **Approach note:** part 1 (story to numbers) for the network and tour data. **Proposes for Iteration 2:** module B (route planning).
Hasnat Ahmed 23i-2652 AI-B	**Data modules (generator):** crates and capacity (category quotas per study), rescue windows (planted touching pairs, clustered windows for study 3), incidents (arrival rate, aftershock bursts), pipeline_dims and drum_prices. Traps 46 (dock-order) and 47 (best-looking-crate), and variety rule 1. **Validator checks:** V4 connectivity and waves; V6 planted tour; V7 signature rules; V8 trap 48. **Claims:** cargo_opt, window_count_opt, window_value_opt, drum_opt. **Approach note:** part 3 (the traps). **Proposes for Iteration 2:** modules A (relief inventory) and C (rescue scheduling).
Hussain Muhammad Ibrahim 23i-2626 AI-B	**Data modules (generator):** cases sheet and workbook writer, radio_log, radio_patterns and radio_reports, sensor graph with planted cover, safety rules with planted assignment, rations with planted selection. Trap 49 (near-miss), and variety rules 3 and 4. **Validator checks:** loader and framework; V1 structure, V2 ranges, V3 identifiers; V6 cover, assignment and rations; V5 claims re-check; V8 trap 47. The broken test copies and validation_report.txt. **Claims:** cover_min. **Approach note:** part 2 (planted objects and claims). **Proposes for Iteration 2:** module D (emergency queue).